



NAME : HAMDAN ALI

WEEK 5 : Day 22

DAY 22

Create reusable componenets like Header and usecard and pass data using props.

Task : Screenshoots Of Output

CODEBASE Of the Work

The screenshot displays a code editor interface with a sidebar on the left showing the project structure. The main editor area shows the content of the `App.css` file, which defines a root object with various color and background properties. Below the code editor, there is a terminal window showing the output of the Vite development server, including the local URL and a list of file updates.

Project Structure (Left Sidebar):

- INTERNSHIP (HINDUKUSHSOFT TECHNOLOGIES)
 - Week1
 - Week2
 - Week3
 - Week4
 - Week5
 - Day21
 - Day22
 - node_modules
 - src
 - components
 - Footer.jsx
 - Header.jsx
 - SearchBar.jsx
 - SkillBadge.jsx
 - UserCard.jsx
 - data
 - teamData.js
 - App.css (selected)
 - App.jsx
 - main.jsx
 - index.html
 - package-lock.json
 - package.json
 - Task22.doc
 - vite.config.js
 - Day23
 - Day24
 - Day25
 - node_modules

App.css Content (Main Editor):

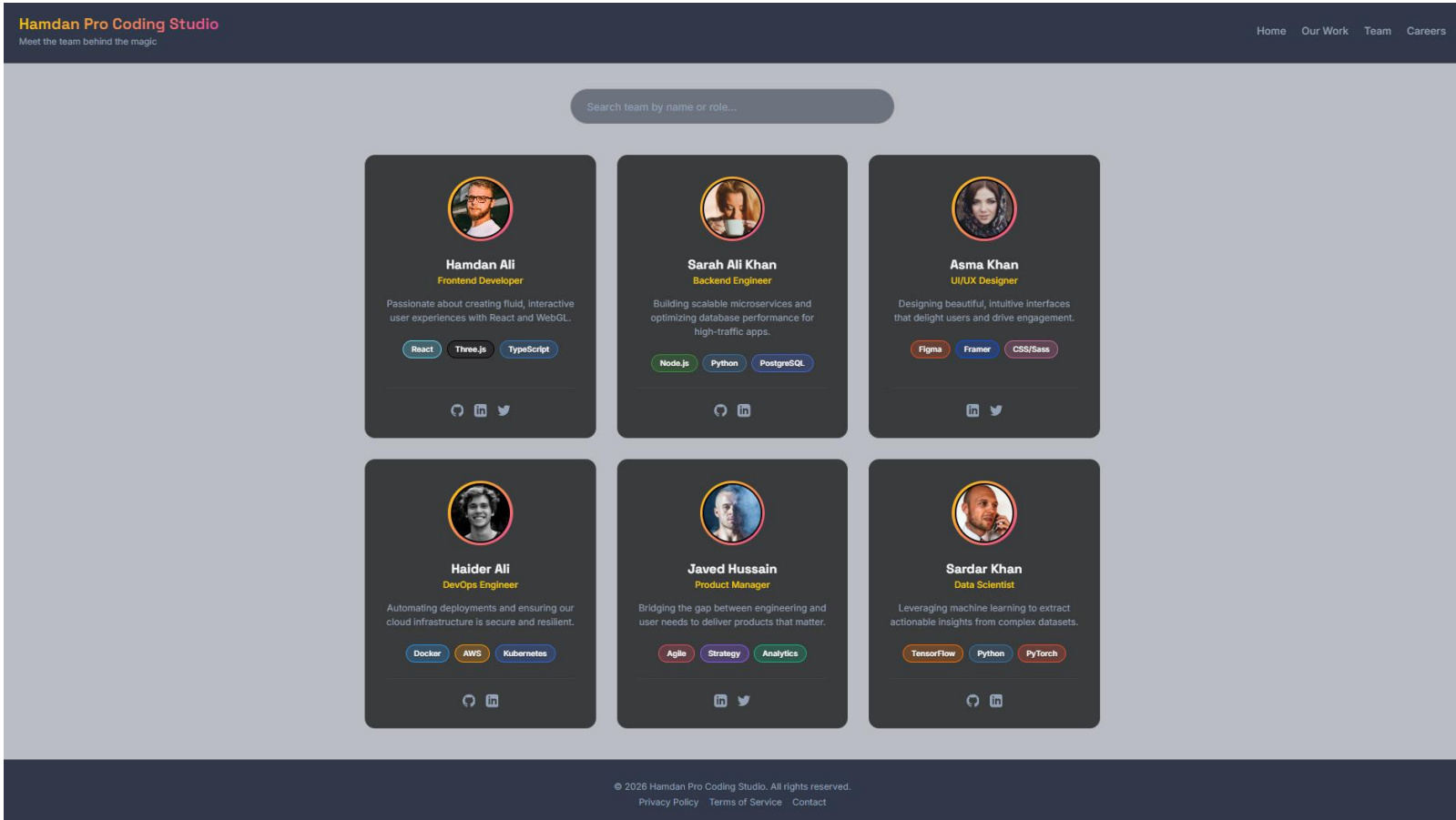
```
Week5 > Day22 > src > # App.css > :root
1  :root {
2    --bg-primary: #babdc3;
3    --bg-secondary: #050505;
4    --bg-card: rgb(0, 0, 0);
5    --text-primary: #F8FAFC;
6    --text-secondary: #94A3B8;
7    --accent-1: #fdcc0b;
8    --accent-2: #8B5CF6;
9    --accent-3: #EC4899;
10   --success: #10B981;
11   --warning: #F59E0B;
12   --danger: #EF4444;
13 }
```

Terminal Output (Bottom):

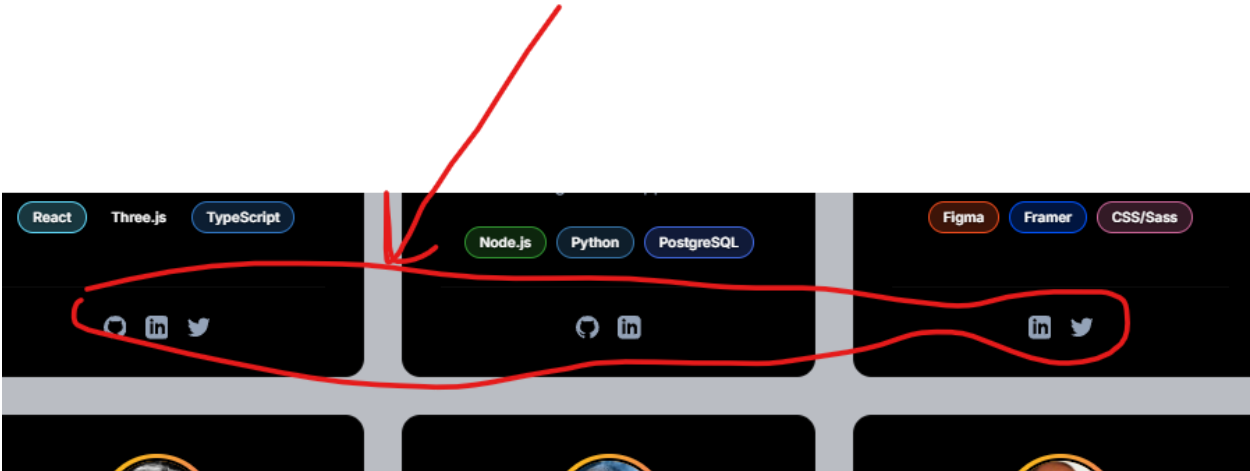
```
PS D:\tasks\Hamdan(FullStack_Internship_At_HindukushSoft Technologies)\internship (Hin
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
1:24:47 PM [vite] (client) hmr update /src/App.jsx
1:25:12 PM [vite] (client) hmr update /src/App.jsx (x2)
1:25:37 PM [vite] (client) hmr update /src/App.jsx (x3)
1:25:40 PM [vite] (client) hmr update /src/App.jsx (x4)
1:26:17 PM [vite] (client) hmr update /src/App.jsx (x5)
1:28:10 PM [vite] (client) hmr update /src/App.jsx (x6)
1:28:21 PM [vite] (client) hmr update /src/App.jsx (x7)
1:28:30 PM [vite] (client) hmr update /src/App.jsx (x8)
1:29:09 PM [vite] (client) hmr update /src/App.jsx (x9)
1:30:08 PM [vite] (client) hmr update /src/App.jsx (x10)
1:30:38 PM [vite] (client) hmr update /src/App.jsx (x11)
1:30:56 PM [vite] (client) hmr update /src/App.css
1:31:56 PM [vite] (client) hmr update /src/App.css (x2)
1:32:14 PM [vite] (client) hmr update /src/App.css (x3)
1:32:56 PM [vite] (client) hmr update /src/App.css (x4)
1:33:04 PM [vite] (client) hmr update /src/App.css (x5)
1:36:15 PM [vite] (client) hmr update /src/App.css (x7)
```



Full Task View



When we click on the respective links linkedin ,github etc it go girect to that app or website.





EXPLANATION OF THE PROJECT

DAY 22

Day 22: Reusable Components and Props

React Training Program - Week 5

Introduction and Approach

Today I tackled building a "Meet the Team" directory using React. My main goal was to deeply understand and implement reusable components using props. Instead of building one massive, monolithic component that handles everything, I broke the UI down into small, distinct pieces. This approach not only makes the code cleaner but also allows me to reuse these pieces across different parts of an application.

Setup and Tools Used

I set up the project manually from scratch using Vite. I created the `package.json`, configured `vite.config.js`, and set up the standard React entry points (`index.html` and `main.jsx`). For styling, I stuck strictly to vanilla CSS, utilizing custom properties (CSS variables) to define a cohesive, premium dark theme featuring glassmorphism, smooth animations, and vibrant gradient accents.

Code Walkthrough

Here's a breakdown of the components I built and how they interact:

1. Header Component (`Header.jsx`)

This is a simple presentation component. It takes `title`, `subtitle`, and an array of `navLinks` as props. By passing these values in, the Header isn't hardcoded to just say "Nexus Studio"; I could easily drop it into another page and give it a completely different title.

2. SearchBar Component (`SearchBar.jsx`)

The SearchBar needed to communicate back to the parent. It accepts a `placeholder` string and an `onSearch` callback function. When the user types, the local `onChange` handler fires, passing the current input value back up to the App component via the `onSearch` prop. This is a classic example of "inverse data flow".



3. UserCard and SkillBadge Components

The `UserCard` is the star of the show. It receives a wealth of data via props: `name`, `role`, `avatar`, `bio`, `skills`, and `social links`. Inside, it maps over the `skills` array and renders a `SkillBadge` for each one. The `SkillBadge` itself is incredibly simple, taking just a `name` and a `color` to render a styled pill. This composition—nesting components within components—is what makes React so powerful.

4. App Component (`App.jsx`)

The `App` component acts as the orchestrator. It imports the `teamData` array and maintains the `searchTerm` in its local state. It filters the team data based on the search input, and then maps over the filtered results, rendering a `UserCard` for each person. All the individual components come together here to form the complete page.

Key Learnings

The most important takeaway for me today was mastering how data flows in React. Data always flows downwards (from parent to child) via props. When a child needs to update the parent (like the `SearchBar` updating the search filter), we pass a function down as a prop, and the child calls that function. It feels very elegant once you get the hang of it.

Challenges and Solutions

One challenge was making the glassmorphism effects look premium without relying on external libraries. I spent some time tweaking the `backdrop-filter: blur()` combined with semi-transparent background colors and subtle white borders. I also added staggered fade-up animations to the cards by using the `map index` to calculate an `animationDelay`, passing it as a style prop to the `UserCard`. The result is a very smooth, modern feel as the page loads.

Conclusion

I'm really proud of how this turned out. The code is modular, the UI looks stunning, and I have a firm grasp on how to build and compose reusable components using props. I feel ready to tackle more complex state management next!

THANK YOU!